FULL-TEXT SEARCH ENGINE ON A SUFFIX-ARRAY INDEX

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

M. Sai Chaitanya (2520090023)
D. Bala Charan (2520090087)
Team Number: 16

Section: 10

Under the guidance of

Ms. K. Chandusha

Asst. Prof., CS&IT

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

DECLARATION

We hereby declare that the project report entitled “Full-Text Search Engine on a Suffix-Array Index” is a record of bonafide work carried out by M. Sai Chaitanya (2520090023) and D. Bala Charan (2520090087), Team 16, Section 10, in partial fulfilment of the requirements for the course Data Structures and Algorithms -3 (25CS2103E) during the academic year 2026-2027.

The work presented in this report is based on the project abstract and project presentation supplied by the team. The report has been organized according to the departmental project-report format and does not claim measured implementation results where such measurements were not included in the supplied materials.

M. Sai Chaitanya – 2520090023

D. Bala Charan – 2520090087

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project report entitled “Full-Text Search Engine on a Suffix-Array Index” is a bonafide work done and submitted by M. Sai Chaitanya (2520090023) and D. Bala Charan (2520090087) in partial fulfilment of the requirements for the course Data Structures and Algorithms -3 (25CS2103E), in the Department of Computer Science and Information Technology, K L (Deemed to be) University, during the academic year 2026-2027.

Signature of the Supervisor	**Signature of the HOD**	**Signature of the External Examiner**

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance, encouragement and support received throughout the work. We express our sincere gratitude to everyone who contributed directly or indirectly to the completion of this project.

We express our deepest appreciation to our guide, Ms. K. Chandusha, Asst. Prof., CS&IT, for her valuable guidance, encouragement and support during the development and documentation of the project.

We also thank the Department of Computer Science and Information Technology and the university for providing the academic environment and facilities required to undertake this Data Structures and Algorithms - 3 project.

Finally, we thank our parents, friends and classmates for their encouragement, cooperation and support during the completion of the project.

M. Sai Chaitanya – 2520090023

D. Bala Charan – 2520090087

TABLE OF CONTENTS

S.No.	Contents	Page No.
1	Abstract	6
2	Introduction	7
3	Literature Survey	8
4	Problem Statement and Objectives	10
5	Hardware and Software Requirements	11
6	Implementation	12
7	Results & Discussion	13
8	Conclusion & Future Scope	14
9	References	15

1. ABSTRACT

The Full-Text Search Engine on a Suffix-Array Index is designed to provide fast and efficient searching over large collections of text documents. Unlike traditional keyword-based search methods, the system supports substring, prefix and phrase queries, enabling flexible information retrieval. The system constructs a suffix array in $O(n \log^2 n)$ time and computes the Longest Common Prefix (LCP) array using Kasai's Algorithm in $O(n)$. Queries are processed through binary search with LCP acceleration, reducing repeated character comparisons and supporting efficient substring matching. Search results are ranked according to relevance and frequency of matching terms, improving the usefulness of returned results. The project presentation compares the proposed suffix-array approach with linear search, inverted indexes and suffix trees. The suffix-array design offers lower memory use than pointer-heavy suffix trees and provides arbitrary substring support without relying on tokenization. The project is intended for applications such as digital libraries, search engines, bioinformatics and text analytics. The supplied project materials describe the expected advantages and system pipeline; they do not provide measured benchmark values or application execution screenshots, so this report presents the documented design and expected outcomes without inventing performance measurements.

Team Number: 16

Team Members: M. Sai Chaitanya (2520090023)

D. Bala Charan (2520090087)

2. INTRODUCTION

2.1 Background

Modern applications frequently need to search large collections of textual information. Exact keyword search is useful for many tasks, but it is less suitable when a user needs to find an arbitrary substring, a prefix, or a phrase occurring inside a document. The project presentation identifies linear scanning as too slow for large collections and notes that inverted indexes are optimized for keywords, while suffix trees can support substring search at the cost of substantially higher memory use.

2.2 Motivation

The motivation of this project is to design a compact and efficient text-search index that can handle more flexible query types while keeping memory usage lower than suffix-tree approaches. A suffix array stores sorted suffix positions rather than a pointer-heavy tree structure. Combining the suffix array with an LCP array enables faster binary-search comparisons.

2.3 Proposed Approach

The proposed system collects a text corpus, preprocesses and normalizes the text, constructs a suffix array, computes the LCP array using Kasai's Algorithm, searches the indexed text using LCP-accelerated binary search, ranks the matching results, and displays the ranked results with positions and scores.

2.4 Scope

The documented scope includes substring, prefix and phrase searching over text documents or a text corpus. The project presentation identifies digital libraries, search engines, bioinformatics and text analytics as target application domains.

2.5 Organization of the Report

The remaining report presents a comparative literature survey, the problem statement and objectives, hardware and software requirements, the implementation modules, results and discussion, and the conclusion with future scope. References are limited to the materials supplied for this project because no external URLs were included in the source documents.

3. LITERATURE SURVEY

The following literature survey reviews **8 existing projects, systems, and research implementations** related to full-text search, suffix arrays, suffix indexes, and compressed text indexes. These works were selected because they address problems similar to the proposed **"Full-Text Search Engine on a Suffix-Array Index"**, particularly substring searching, efficient indexing, memory usage, and large-scale text retrieval. The proposed project combines ideas from these approaches by using a suffix array with an LCP array and LCP-accelerated binary search. The project abstract and PPT specifically identify suffix arrays as a compact alternative to suffix trees and as a solution for flexible substring, prefix, and phrase searching. DSA-3_Project Abstract Full-Text-Search-Engine-on-a-Su…

3.1 SAES – Full-Text Search Engine with Suffix Index

SAES is a full-text search system designed for searching large and heterogeneous text collections using a **suffix index** instead of a conventional inverted index. The system uses suffix arrays as its core indexing structure and integrates suffix-index technology with Elasticsearch and Lucene. It supports flexible searches and is designed to operate on large-scale data using distributed index construction and merging. This project is highly relevant to the proposed system because it demonstrates the practical use of suffix arrays for full-text search and addresses the limitations of token-based inverted indexes.

SAES – Full-text search engine with suffix index

3.2 Distributed Text Search Using Suffix Arrays

The **Distributed Text Search Using Suffix Arrays** project/research work investigates how suffix arrays can be distributed across multiple machines for searching very large databases. It focuses on the difficulty of storing and accessing a large suffix array when the complete index cannot efficiently fit into the memory of one machine. The work studies how suffix-array partitions can be organized across a cluster while maintaining efficient query processing. This approach demonstrates that suffix arrays can be extended beyond single-machine applications to support large-scale distributed text-search systems.

Distributed Text Search Using Suffix Arrays

3.3 Apache Lucene Suffix-Array Search

Apache Lucene is a widely used Java-based full-text search library that provides indexing and searching capabilities for applications. A Lucene project proposal explored the use of **suffix arrays for improving searches with leading wildcards**, where conventional dictionary-based searching can become inefficient. The proposed suffix-array approach was intended to improve performance for queries beginning with wildcard characters while preserving search results. This work is relevant because it shows how suffix-array techniques can complement traditional inverted-index search engines for difficult substring-like queries.
Lucene Suffix Array Search Proposal

3.4 FM-Index Engine

The **FM-Index Engine** is an educational implementation of modern full-text indexing techniques based on suffix arrays. It implements suffix-array construction using the DC3/Skew algorithm, followed by the Burrows-Wheeler Transform and FM-index construction. The system supports efficient substring searching using backward search and aims to demonstrate how suffix arrays can be transformed into a more compact full-text index. This project is relevant to the proposed work because the FM-index is built on suffix-array

concepts and demonstrates another method of achieving efficient substring search with reduced memory usage.

FM-Index Engine – GitHub Project

3.5 Dynatrace Index4J – FM-Index Library

Index4J is an FM-index implementation that provides compressed full-text substring indexing using suffix arrays, the Burrows-Wheeler Transform, bit vectors, and wavelet trees. It supports operations such as counting pattern occurrences and locating the positions of matching patterns without requiring prior tokenization. The project demonstrates how suffix-array-based structures can provide flexible substring searching while keeping storage requirements relatively low. It is closely related to the proposed project because both approaches support arbitrary pattern searching without depending entirely on word-based indexing.

Dynatrace Index4J – GitHub Project

3.6 Comparative Summary of Literature

S.No.	Existing Project / System	Main Technique	Relevance to Proposed Project
1	SAES	Suffix Index / Suffix Array	Full-text search over large heterogeneous data
2	Distributed Text Search	Distributed Suffix Arrays	Large-scale suffix-array searching
3	Apache Lucene Suffix Array	Suffix Array + Lucene	Faster wildcard/substring-like search
4	FM-Index Engine	Suffix Array + BWT + FM-index	Efficient substring searching
5	Index4J	FM-index + Wavelet structures	Compact full-text substring search

3.10 Research Gap and Proposed System

The reviewed systems show that suffix arrays and their derivatives are effective for flexible full-text searching. However, several approaches introduce additional complexity through distributed systems, FM-index structures, wavelet trees, or learned models. The proposed project takes a comparatively straightforward approach by combining a **suffix array with an LCP array and LCP-accelerated binary search**. According to the project PPT, the suffix array is constructed in $\mathbf{O(n \log^2 n)}$ time, while Kasai's Algorithm constructs the LCP array in **O(n)** time. This provides a practical balance between flexible substring searching, query efficiency, and memory usage.

4. PROBLEM STATEMENT AND OBJECTIVES

4.1 Problem Statement

Develop a full-text search engine that can efficiently search large text collections for substring, prefix and phrase queries using a compact suffix-array index.

4.2 Problem Description

Traditional linear search is slow on large corpora, while keyword-oriented inverted indexes do not naturally support arbitrary substring queries. Suffix trees support substrings but have higher memory and structural complexity. The project therefore uses a suffix array with an LCP array to balance flexible search capability, query speed and memory usage.

4.3 Objectives

- Enable fast search over large document collections.
- Support substring, prefix and phrase queries.
- Construct the suffix array in $O(n \log^2 n)$ time.
- Construct the LCP array using Kasai's Algorithm in $O(n)$.
- Accelerate query processing using LCP information during binary search.
- Rank matching results according to relevance and term frequency.
- Keep memory usage lower than suffix-tree-based indexing.

4.4 Expected Benefits

- Faster search than linear scanning for large corpora.
- Flexible matching beyond exact token-based keyword queries.
- Lower memory requirements than suffix trees.
- Competitive query performance for the supported query types.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

Component	Requirement
Memory	4 GB minimum
Storage	1 GB or more

5.2 Software Requirements

Software / Item	Requirement
Operating System	Windows / Linux
Programming Language	Java
IDE	VS Code / IntelliJ IDEA
Runtime	Java Runtime Environment (JRE)
Input	Text documents / corpus
Output	Ranked search results

5.3 Core Algorithms and Techniques

Technique	Role in the System
Suffix Array	Sorted suffix positions used for efficient pattern matching through binary search.
LCP Array + Kasai's Algorithm	Computes longest common prefixes in O(n) and reduces repeated character comparisons.
LCP-Accelerated Binary Search	Uses LCP information to narrow comparisons during query processing.
Text Preprocessing & Ranking	Normalizes the input text and ranks matches using frequency and relevance.

6. IMPLEMENTATION

The implementation is organized into the modules described in the supplied project presentation. The descriptions below document the intended system pipeline; exact source-code listings were not included in the supplied files.

6.1 Module 1 – Text Collection and Preprocessing

The system first gathers the target documents or corpus. The input is then cleaned and normalized before indexing. The presentation describes this stage as text preprocessing and notes that the input is a collection of text documents or corpus data.

6.2 Module 2 – Suffix Array Construction

The suffix-array module generates suffixes of the text, sorts them, and stores their starting positions. The documented construction complexity is $O(n \log^2 n)$. The sorted suffix positions provide the ordered search space required for binary-search-based pattern matching.

6.3 Module 3 – LCP Array Construction and Accelerated Search

The LCP module uses Kasai's Algorithm to compute the longest common prefix between adjacent suffixes in O(n). During query processing, LCP information is used to reduce repeated character comparisons in binary search. This supports efficient substring, prefix and phrase matching.

6.4 Module 4 – Ranking and Result Display

After matching suffix positions are identified, the system ranks results using relevance and term frequency. The output is a set of ranked search results that includes positions and scores according to the project methodology.

6.5 Implementation Pipeline

Step	Module	Operation
1	Text Collection	Gather target documents or corpus data.
2	Text Preprocessing	Clean, normalize and prepare input for indexing.
3	Suffix Array Construction	Generate and sort suffixes; store starting positions.
4	LCP Array Construction	Use Kasai's Algorithm to compute adjacent suffix LCP values.
5	Query Processing	Perform LCP-accelerated binary search for the query.
6	Result Ranking	Rank matches using relevance and term frequency.
7	Display Results	Return ranked matches with positions and scores.

7. RESULTS & DISCUSSION

The supplied project presentation describes the expected system outcomes but does not contain application-run screenshots, measured query times, corpus sizes, or numerical benchmark tables. Accordingly, this section reports the documented functional outcomes without presenting invented experimental measurements.

7.1 Functional Outcomes

Outcome	Discussion
Fast Search	The suffix-array index is intended to provide much faster querying than a linear scan on large corpora.
Flexible Matching	The system supports substring, prefix and phrase queries.
Efficient Indexing	Suffix-array construction is specified as $O(n \log^2 n)$, while LCP construction using Kasai's Algorithm is $O(n)$.
Reduced Memory	The suffix-array representation uses less memory than a pointer-heavy suffix tree.
Relevance-Based Results	Matches are ranked using term frequency and relevance.

7.2 Comparison with Other Approaches

Approach	Main Characteristic	Discussion
Linear Search	Repeated scans for every query	Expected to be slower on large corpora
Inverted Index	Fast token-based keyword lookup	Less suitable for arbitrary substring search
Suffix Tree	Fast substring support	Higher memory usage
Suffix Array + LCP	Binary search with LCP acceleration	Chosen for flexible search with lower memory use

7.3 Result Interpretation

The project design addresses the central problem identified in the presentation: obtaining flexible text matching without the high memory cost associated with suffix trees. The LCP array is important because it reduces repeated character comparisons during binary search. The ranking stage further improves usability by ordering the matching results according to relevance and frequency.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Full-Text Search Engine on a Suffix-Array Index provides a structured approach to searching large text collections. The project combines suffix-array indexing, Kasai's LCP construction, LCP-accelerated binary search, text preprocessing and relevance-based ranking. Based on the supplied abstract and presentation, the design supports substring, prefix and phrase queries while targeting lower memory use than suffix trees and faster searching than linear scans.

The documented objectives are reflected in the proposed pipeline: text is collected and preprocessed, suffix positions are indexed, LCP information is computed, queries are accelerated through binary search, and results are ranked before display. Since the supplied materials do not contain measured experiments, this report does not claim specific numerical improvements.

8.2 Future Scope

- Evaluate the implementation on larger and more diverse text corpora using measured query-time and memory benchmarks.
- Extend the ranking model beyond term frequency and basic relevance to support richer scoring strategies.
- Improve preprocessing for larger collections and additional text formats.
- Explore parallel or optimized suffix-array construction for large-scale datasets.
- Develop a user-facing search interface suitable for digital libraries or other target application domains.
- Compare the implemented system experimentally with inverted indexes and suffix trees using common benchmark datasets.

9. REFERENCES

1. Xu, W., Nong, G., et al., **"Full-text search engine with suffix index for massive heterogeneous data," Information Systems, 2022.**
   Reference URL
2. **"Distributed text search using suffix arrays."**
   Reference URL
3. Apache Lucene, **"Apache Lucene – Java Full-Text Search Engine."**
   Reference URL
4. Apache Software Foundation, **"Use Suffix Arrays for fast search with leading asterisks – LUCENE-7639."**
   Reference URL
5. Hazem Masarani, **"FM-Index Engine – Efficient implementation of suffix array, BWT and FM-index."**
   Reference URL
6. Dynatrace OSS, **"Index4J – FM-Index Library."**
   Reference URL
7. **"Sapling: accelerating suffix array queries with learned data models." Bioinformatics, Oxford Academic.**
   Reference URL
8. **"An optimized FM-index library for nucleotide and amino acid search."**
   Reference URL
9. **"Approximate String Matching with Compressed Indexes." Algorithms.**
   Reference URL
10. **"INFINI-GRAM MINI: Large-scale FM-index based text search." EMNLP 2025.**
    Reference URL